KV6013 Computing Project – Jamie Clark

Emulator for the MN103 architecture. Supervised by Nauman Aslam

BSc in Computer Science, year 3

jamie.clark

Contents

Figures

Tables

Abstract

CPU emulation has long been an area of interest in computer science and cybersecurity fields. An ideal CPU emulator will be able to run native machine code from its given architecture on any machine, enabling an individual to analyse and use code while bypassing the need for the real, physical device. Emulation is also useful for preserving hardware; long after old CPU models go out of use, there will still be software repositories allowing you to reproduce their behaviour and run old programs from your home computer. Using a systematic testing approach and development methodology, with tools informed by previous emulators and emulator literature such as the CMAKE build system and the C++ programming language, this project built an emulator for one such architecture; the Panasonic MN103.

Chapter 1 - Introduction

The MN103 was previously deployed in video players and cameras and setup boxes, but has fallen out of use [28], despite this, it still sees use in these fields in old models but will not be in use for much longer as these devices become scarcer and scarcer. For preservation, a functional CPU emulator enabling analysis and execution of code for this model has been built with publicly available documents and other open-source emulators as reference. This will preserve and document the architecture long after the physical model leaves industrial use. This involved investigating emulation techniques of other open-source emulators and a systematic approach to development using extensive unit and product testing, as CPU emulation is notoriously error prone.

Chapter 2 - Research and planning

With the stated goal in mind, there are several key components that will require implementation:

- Instruction decoder
- Emulator
- Memory management unit

An instruction when fed to the CPU is simply a stream of arbitrary bytes, with each byte and bit encoding meaning and intention of the instruction as well as any arguments it may take. A real CPU will take these bytes, and will decode them, building a complete picture of what the instruction is, and what exactly it should be doing before executing the corresponding operations. To correctly emulate an instruction, we must also do this in the same way the real CPU does, we take a series of arbitrary bytes, attempting to decode them into a valid instruction.

When the instruction is decoded, the CPU will then execute a series of operations performing the behaviour of this instruction. This could be writing to/reading from registers or memory, for example. We also do this, performing operations on the registers and memory resources we have created in our emulated context. This involves faking memory and a series of registers to ensure the code performs as expected and can operate on the desired resources.

Memory is one of these resources. In a CPU, memory accesses go through several layers of abstraction (from virtual to physical memory, for example) and are then passed off to the memory management unit [4] where the operations are performed on real RAM. In the interest of ensuring consistent behaviour in the emulated environment, we create our own memory management unit, enabling writing and reading of memory in the program for all instructions that need it. Memory permissions are also supported, ensuring expected behaviour when dealing with different sets of permissions – readable, writable, and executable.

These 3 components – decoder, emulator, and MMU will provide complete ability for instructions to do whatever they need to, acting identically to those on the real processor, with some caveats discussed later.

Literature review

An emulator has many incarnations which attempt to do different things, the paper [1] discusses different approaches with some emulating strictly hardware peripherals, some mixing software and hardware, and some even attempting to emulate an entire operating system. Simply emulating the CPU requires a combination: In the context of this project, it was decided to pursue the mixed software and hardware approach; only the hardware & software components necessary to enable instruction behaviour and execution were implemented.

An emulator may function as expected even without implementing some internal components of the CPU, if the execution environment itself is replicated. For example, some behaviours, like the instruction cache are heavily used by the CPU but are transparent to the user [3] – given that it makes no difference to actual instruction behaviour it can be omitted in emulation, although implementations may use their own caching mechanisms to reduce time taken performing expensive operations.

QEMU is an example of an emulator which emulates an entire operating system [2], featuring many subsystems to do this. On top the CPU emulator itself – which is what this project attempts to be there is hardware peripherals, exception support, interrupts, dedicated usermode and kernel mode support among others. Of these systems this project attempts to perform only a small subsection – however for complete system emulation all essential peripherals must be present.

Interpreters

The model used for fetching and decoding in this project is described in [1] as interpreting. An interpreter is a more basic form of emulation in which we simply get instruction bytes, decode them and execute corresponding operations, as mentioned. Interpreters have a notable speed trade-off compared to other some emulation methods [1] as operations are performed in the higher-level language used for the interpreter rather than at the machine level.

```
bool Emulator::handle_not(const Instruction &ins) {

  ArgKind src = ins.kinds[0];
  reg_type s = regs.get(src);

  s = ~s;

  regs.set(src, s);
  // Must also set flags
  reg_type psw = regs.get(ArgKind::PSW);
  psw &= ~0b1111;
  bool cnd = (int)s < 0;
  psw |= (PswBits::N * cnd);
  cnd = s == 0;
  psw |= (PswBits::Z * cnd);

  regs.set(ArgKind::PSW, psw);

  return true;
}
```

Figure 1

Figure 2

Figure 3

Figure 4: Instruction operations for the NOT instruction in the project

In addition to this, writing an interpreter also sacrifices portability [1]; to port to additional architectures large portions of the instruction logic and decoder may need to be reimplemented for the new target architecture.

Dynamic translation

One alternative to the interpreter is through dynamic translation [2]. This is the approach used by QEMU – with the idea being to decompose individual instructions from the target into sets of micro-operations such as moving or adding. These micro-operations can then be translated into the host machine language and executed as such. This approach is much faster than interpreting [1] as it enables operations to happen at the machine level. This translation process may end up slowing down emulation, however this can be mitigated somewhat by caching previous instructions and re-using them whenever they are encountered [2].

Dynamic translation features several notable downsides when compared to interpreting, however. An interpreter is quite easy to debug; instruction handlers can be easily hooked, and it is very easy to

trace which handler comes from which instruction – this is not so clear in dynamic translation, however, as a micro-operation may be difficult to map to its source instruction. It is also harder to work with self-modifying code, as the intention with binary translation is once an instruction is decoded it won't change and can be re-used many times due to that fact; if an instruction changes this assumption is invalidated and it must again be decoded and translated. An interpreter does not have this problem as decoding is performed for every instruction after it is fetched [1], although an interpreter could implement similar caching behaviour.

This and ease of debugging in mind the traditional interpreter approach was preferred over dynamic translation for this project. Being easily debugged and familiar was incredibly important as any bugs encountered or confusions had would bottleneck development work, so it was important to minimize these.

Memory management

For any kind of emulation, the memory management unit is an essential inclusion, not only to replicate the execution environment as accurately as possible but also to provide memory facilities to instructions which need them. When designing memory management for emulation there are some things one must keep in mind. The real, physical memory layout has little in common with the normal virtual address space of a process [1], in the real memory, some regions map to hardware and peripherals (memory mapped I/O, for example) and the book [1] remarks that to run any kind of programs in the emulator this behaviour must be replicated.

Although this is not wrong, it depends on what the emulator seeks to do. Does your emulator aim to provide a fully identical runtime environment in which one can run extremely complex programs such as kernels and entire operating systems with access to hardware peripherals? In the case of this project given the scope and intention this facility is not provided – hardware peripherals and other functionality is out of scope, and the emulator only concerns itself with replicating instruction behaviour. Given the goal, creating an environment not dissimilar to a user space program with virtual memory was fit for purpose.

The MMU exists for this purpose [1]. It provides a layer of abstraction atop the physical memory, organizing memory into pages – chunks of memory the size of page size, which depends on OS CPU and OS, but is often 4kb or 8kb [1]. Each process, along with the main OS process is provided with its own virtual memory space, which protects processes from each other and allows them to have their own resources. The mapping between physical pages and pages in the virtual address space is kept in page tables which store information about the memory inside, along with the real (physical) page address associated with a virtual address [4].

Book [1] comments on the difficulty of correctly emulating the MMU due to its complexity and number of features, but as mentioned earlier in [3], processes which are transparent to the execution environment need not be reproduced in the emulator. It is not necessary to reproduce, for example, the caching mechanisms of the MMU in software as these make no difference to the execution process and operations – besides speed, of course.

With this and the simplified memory layout in mind, developing the MMU for this use case becomes simple; we simply need a way of mapping physical to virtual pages from the perspective of the emulated code, limiting access to these pages based on permissions and faulting for pages which aren't present. This helped inform the development process for this component.

Testing methodologies

Given how error prone emulator development can be it is important to utilize an effective testing methodology [1]. Paper [3] proposes a technique for fuzzing and testing CPU emulators via executing a series of instructions and comparing state – a combination of memory and registers. Although, using

the entire output state is unnecessary, if operations in the program coalesce and cause a consistent output state – and this output state is produced in the same manor this is all that is necessary. We opt for a similar approach, utilizing the final state of the program for comparison with a sane result produced from native code. An additional avenue for testing CPU emulators is simply using a coverage guided fuzzer such as AFL++ [16]. AFL++ is an extended version of the AFL (American fuzzy lop) fuzzer which generates random inputs for a program, then using instrumentation it has applied to the source code is able to track how far into the code a certain input flows (the coverage). Then based on this coverage it can choose different transformations on different parts of the input to expand this it's reach. Paper [3] proposes a more specialized approach which is more efficient than transforming random parts of the input, but given the simplicity of the instruction grammar – just being a stream of bytes, it is relatively easy to achieve high coverage with something like AFL with simple input transformations.

In addition to this we compile several testcases from the c-testcases git repository [5] to run inside the emulator. Each of these tests returns 0 at the end of its execution through a series of operations. Using a combination of this and randomly generated c code we can evaluate and test the emulator to ensure it is operating as expected. In the testing procedure used in [3], machine code and data is randomly generated and mapped into memory, this generation is also fine-tuned by testing each random sequence through the CPU to assess whether it maps to a valid instruction or not. Although we do not generate random instructions for the final evaluation a naïve version of this technique was used sparsely during early testing of the instruction decoder.

Deploying these strategies against the real CPU is a useful and effective approach for finding bugs and differences [3], however it is not the only technique used. Some CPUs are difficult to get a hold of and expensive to set up and environment for, with this being the case another method used is testing the emulator against a different, trusted CPU core [1]. This has some downsides, for example some code sequences may produce different results purely due to architectural differences such as word size or processing speed (in the case of multithreaded code) – however if code is generated and tested in a cross-platform way and only the final value is tested the output values should be the same. For example, given portable c code which computes the Fibonacci sequence using 32bit integers is compiled and executed for both x86 and the MN103 respectively, the outputs should be the same whichever binary is executed. This is not entirely reliable as outputs may be the same precisely because of a bug, however the possibility of this can be minimized by ensuring outputs are consistent across many programs.

Legality and preservation

A key sticking point with emulation in some circles is the legal stigma. Many emulator developers have historically been pursued with legal threats. A key example of which being the numerous conflicts Nintendo has had with developers [9]. Broadly speaking emulation is entirely legal - even if the program in question is used almost entirely to play pirated, copyrighted material if the developers themselves do not point users in the direction of pirated media, bypass copyright protections or break encryption [9]. This of course makes sense – you cannot copyright the behaviour of a system. A demonstration of this is the practice of Clean Room design, in which a system's behaviour is recreated via first reverse engineering the system and then reproducing the behaviour without infringing on copyright, generally by having a development environment which is uncontaminated by proprietary techniques used by the copyright holders [12].

Some cases, however, are quite justified. It's hard to ignore the association between emulation and piracy. Emulators are often used to run pirated material, especially those made to emulate for videogame consoles. Some emulators fail to distance themselves effectively from this material, with some actively linking to it and encouraging it [9].

That said – it's important to highlight the value of emulation especially in preservation. The MAME project [8] seeks to preserve old videogame consoles and hardware, saving them from being lost and forgotten. MAME serves as an example of the profound good emulation can do for preserving and documenting technology. It, and other open-source emulators in general also serve as a way of documenting features and behaviours of hardware.

The MAME project and others explicitly state the purpose of their emulator on the website and require that ROM files for games must be supplied by the user, and state in clear terms that no copyrighted material is supplied by the creators, and to use the system original copies of ROMs are required [8]. Also stated on this page is the intention for individuals not to link to illegal ROMs, CDs, or media files, building a defence and specifying intent if a company was to falsely identify the project as piracy enabling.

A tactic employed by some companies such as Nintendo is to introduce layers of encryption to their devices to not only make the process of emulation more difficult but also make it harder to perform without breaking the DMCA copyright act, which states that distribution of tools which would bypass these layers is copyright infringement. One such case was the Yuzu emulator built for the Nintendo switch which shared information on obtaining decryption keys, and because of this settled in court and removed the emulator. After this happened, forks of the Yuzu project appeared which did not share this information which are perfectly legal and still up today [17].

Security and vulnerabilities

A facet of emulation not considered as much as the legal implications, or the goal of the development process is security. Emulators such as QEMU have long been targeted with exploits which seek to escape the emulator and execute code on the host machine [18], this is due to emulators use in security critical fields such as malware analysis where behaviour of a program must be inspected but cannot affect the host machine. Such exploits target vulnerabilities in source code areas which parse and process user input in a complex manner, such as instruction decoding and emulation components, and in code implementing behaviour for peripherals and devices [22, 23, 24]. To address these vulnerabilities and help avoid further issues projects like QEMU use secure coding principles [25] and fuzzing of their project to identify inputs and behaviours which trigger bugs and subsequently be exploited.

Speed and performance

Emulation speed is another massive concern [1], as such it makes sense to take advantage of speed and caching features of the host CPU, compiler, and programming language used to maximise performance in the target. One such feature provided by C++ is the constexpr specifier [19]. This applies to expressions and functions and where they satisfy the criteria allows them to be evaluated at compile time. For functions this means that if parameters are const or unchanging the function may be evaluated before running and the result will be embedded in the binary, avoiding unnecessary runtime cost. This can lead to significant performance gain in performance critical applications [20]. Another such feature is the likely/unlikely attribute pair introduced in C++20. These can be used to label blocks of code with diverting paths and allow the compiler to optimize based on the most likely path. An example of this is loop code which contains a path which only triggers at specific intervals which make up a very short amount of the loop's execution time - naturally this code wouldn't need to be considered in the "hot" path and could be positioned elsewhere in the function or loop to stash it out of the way. Use of these attributes should be cautious however as the C++ standard notes: "Excessive usage of either of these attributes is liable to result in performance degradation" [21].

In conclusion for an emulator to be useful to users it must reliably and consistently reproduce the targeted environment or component of the target environment, providing all necessary features and peripherals. It should provide speed and performance while preserving security and be able to handle

arbitrary user input without causing unexpected behaviour. To further the cause of security and consistency within the emulator it should also be easily testable, as this will be crucial to weed out bugs and vulnerabilities.

Requirements

After examining existing projects and literature the following specification for requirements was developed:

Functional Requirements

Table 1

ID	Name	Description	Priority
F01	Complete coverage	The emulator must be able to handle every instruction present in the MN103 instruction set	Must
F02	Debugging	The emulator must contain detailed debugging logs and information pertaining to the execution of instructions and changes in registers and memory.	Must
F03	Emulator component	The emulator component must be present and handle instruction output from the decoding component, reproducing the expected behaviour of instructions.	Must
F04	Decoder component	The decoder component must be present and handle parsing of the instruction input into instruction instances which are passed to the emulator component.	Must
F05	Memory management component	The memory management component must be present and handle memory accesses in emulated code. It must allow reading and writing to arbitrary mapped addresses and where addresses are not mapped or	Must

		accessible it must deny these accesses, producing a notable exception or error.	

Nonfunctional requirements

Table 2

ID	Name	Description	Priority
N01	Reliability	The emulator must reliably parse and execute emulated code consistently	Must
N02	Testability	The emulator should be easy to test.	Should
N03	Consistency	The emulator must consistently produce the same behaviour for identical instructions.	Must
N04	Performance	The emulator should not contain unnecessary performance bottlenecks and should not use excessive amounts of memory.	Should
N05	Capacity	The emulator must be able to handle an arbitrary amount of instruction data and executions while maintaining consistency (N03)	Must
N06	Legal compliance	The emulator must not violate the copyright of any entities involved in the development process.	Must
N07	Security	The emulator should be designed with security in mind; user input is entirely arbitrary and untrusted and should be treated as such.	Should

Tooling

With familiarity and ease of debugging in mind, C++ was chosen as the project language and GCC [27] as the compiler. C++ has long been used effectively to create emulation projects such as dolphin [10], and its features such as OOP and use of the STL meant it was a good choice for this project as they enable clean design; segmenting components via encapsulation and joining them where appropriate using composition – in this case the Emulator contains the Memory manager and Decoder

classes. Use of the STL was another deciding factor as it enabled less time to be spent on implementing boilerplate code and more time for debugging and implementing more complex logic. The debugger chosen was GDB as it is easily scriptable and versatile and easily available on most Linux distributions, while also being familiar.

Chapter 3 - Practical work

The architecture for the program has the Emulator containing the Decoder, Registers, and MMU. The Emulator receives the instructions in the emulator loop, which then attempts to decode the instruction via the decoder, then the corresponding handler for that instruction is invoked, which may make use of MMU and registers, or both.

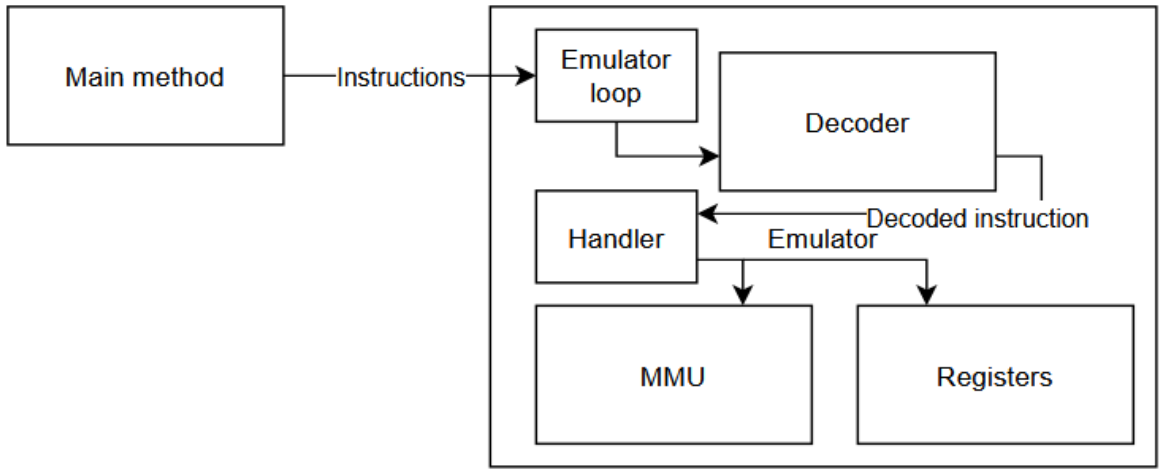


Figure 5: Emulator architecture

This particular structure was chosen as It allowed the emulator to contain all resources it needed access to and outside entities to supply input which could then be processed without the need for visibility into the emulator internals – that said, if an entity needs to change settings, such as the pages mapped in the MMU, they can do so by using the getters present on the emulator class.

Decoder

The purpose of the decoder mirrors that of the decoding step in the CPU fetch-decode-execute pipeline and the decoding modules present in other emulators such as QEMU or ARMY [11], to parse the raw instruction bytes and transcribe them into a format which is easily used by the emulator to perform the instruction's associated behaviour.

The decoder is split up into sections as per the instruction map in the manual provided by Panasonic [26], having a function for each section capable of handling instruction combinations for that section. This approach was chosen as the architecture inconsistently has similar encodings for instruction variants, with some instructions having completely different formats.

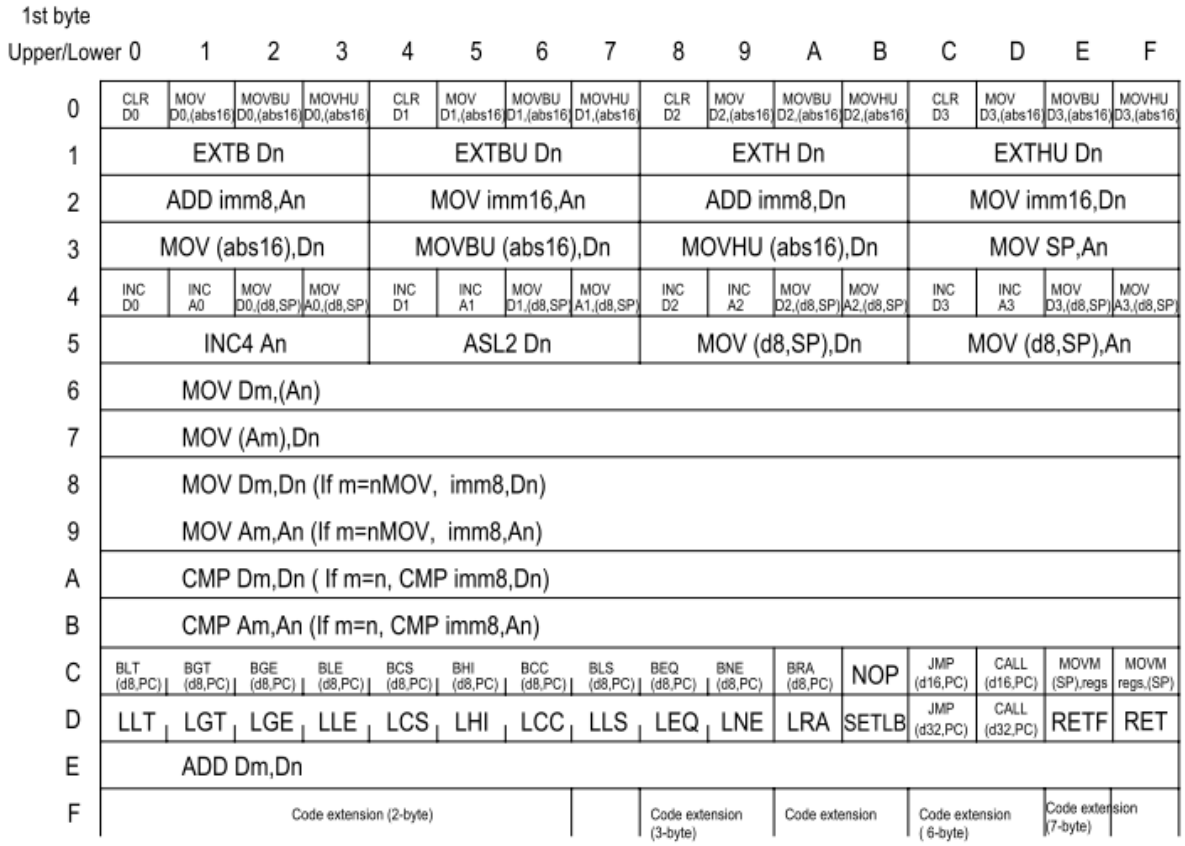


Figure 6: Instruction mapping for Sn types

As shown in figure 2, instruction groups may feature many different instructions with many different argument types, meaning there is few trends that can be relied on to classify instructions, except for consistency of some subgroups - see the 0xD row for example, where Lcc instructions are grouped together.

A similar issue is present when decoding instructions for ARM CPUs, however not nearly to the same degree here as ARM encoding utilises a clear grouping of instructions and purpose based on a few bits.

A64 instruction formats

Type	Bit																														
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	0	op_0		0	0	0	0	op_1																								
SME	1	op_0		0	0	0	0	Varies																								
Unallocated				0	0	0	1																									
SVE				0	0	1	0	Varies																								
Unallocated				0	0	1	1																									
Data Processing — Immediate PC-rel.	op	$immlo$		1	0	0	0	0	$immhi$																					Rd		
Data Processing — Immediate Others	sf			1	0	0	01–11																							Rd		
Branches + System Instructions	op0			1	0	1	op1																							op2		
Load and Store Instructions	op0				1	op1	0	op2			op3											op4									
Data Processing — Register	sf	op0		op1	1	0	1	op2										op3													
Data Processing — Floating Point and SIMD	op0				1	1	1	op1		op2																					

Figure 7: ARM instruction encoding table

Also, in contrast to this, the x86 encoding denotes instruction type using an opcode byte and optional prefix [15], significantly easing efforts to decode instructions.

Fully decoded instructions are represented in a struct with 5 data members:

- Unsigned char sz
- Unsigned char op
- ArgKind kinds[3]
- Unsigned char args[6]
- Unsigned char curr

`sz` is used to denote how many bytes of space the instruction needs, this changes based on arguments and instruction type. `op` determines the instruction type, if the instruction is a MOV, the `op` will reflect this. `kinds` stores a list of max 3 elements denoting the types of instruction arguments. These could be registers or data, and may not even contain 3 elements, as where no more arguments are required the array is terminated with a 0. `args` contain values of arguments, these could be addresses, immediate values, or displacement values applied to registers. There is a maximum of 6 bytes of argument space – no instruction will ever need more than this. Finally, `cur` is used to mark the currently processed argument, it tracks how much of the `args` buffer is being used so no arguments are written out of bounds of the buffer.

Unsigned char is used prevalently here to preserve as much space as possible, as for the `curr` member for example there will never be a need to store anywhere near 256 arguments, let along the capacity of a full 4-byte integer, so a single byte will suffice. An alternative structure would have the opcode byte stored inside the `args`, and simply have `args` contain the entire instruction, however this would complicate processing and make it less explicit what exactly is being extracted from the instruction. Space was a concern here as there could potentially be hundreds of thousands if not millions of instructions, so it was pivotal space be minimized to ensure maximum throughput and minimize memory usage.

Another alternative storage method would have the `args` array dynamically allocated and only have space reserved for exactly what is needed. This, however would have necessitated calling into the memory allocator to adjust the buffer sizes for instructions which would have slowed down processing, as such we contain enough space for all possibilities and don't waste time tampering with it.

Testing procedure

Each map handler was unit tested on processing the instructions it encapsulates to ensure consistency with the expected outcome. The output of the decoding Is then logged so it can be compared against the assembly produced by the assembler. Doing this we ensure that each segment is decoded successfully in isolation and can be relied on to build up to further testing of the emulator.

Emulator

The emulator takes the output of the decoder and performs the behaviour expected for each instruction. This is accomplished through handlers developed for each instruction, with some handlers being re-used due to instruction behaviour differing only slightly – add and sub, for example. This approach was chosen as it enabled a clean and structured approach where logic could be re-used as appropriate with each instruction having its own designated handler to which its behaviour is isolated.

Behaviour for each instruction is outlined inside the manual in the "Instruction Specification" section, with each instruction being composed of a series of operations with their types and operands specified.

MOV imm,Reg

Operation	imm→Reg Moves the contents of the immediate value(imm) to the register(Reg).

Assembler mnemonic	Notes	V	C	N	Z	Size	Cycles
mov imm8,Dn	imm8 issign-extended	—	—	—	—	2	1
mov imm16,Dn	imm16 is sign-extended	—	—	—	—	3	1
mov imm32,Dn		—	—	—	—	6	2
mov imm8,An	imm8 is zero-extended	—	—	—	—	2	1
mov imm16,An	imm16 is zero-extended	—	—	—	—	3	1
mov imm32,An		—	—	—	—	6	2

Flag Changes

VF: No Changes.
CF: No Changes.
NF: No Changes.
ZF: No Changes.

Figure 8: Example instruction spec

The behaviour is described in detail and each variant is specified along with notes and any PSW flag changes that may occur. An alternative method to achieve this would be having a single method containing all logic for each instruction – this would reduce the need for having many handler functions and would make it easy to group similar instructions together however would've reduced the visibility into the codebase and ease of debugging massively.

Another method would be to join the decoding and emulation processes together: instead of having the emulator receive the decoded instruction we would have had the decoder call the emulator handler as soon as it knew what the instruction was. This would've been a bit faster than our approach, however, would've resulted in the joining of the emulator and decoder logic which would have complicated development and ease of debugging – it would've been more difficult to diagnose which component was responsible for a fault.

Testing procedure

Each handler was tested in isolation against the behaviour specified in the manual. This ensured that each instruction had consistent behaviour and could be relied on, which in turn meant that when slotted together they should also function exactly as expected. This involved writing out instructions and variants manually and sometimes scripting to generate more instruction possibilities, which would then be sent to the emulator and their behaviour observed.

MMU

The MMU is used by the Emulator to facilitate memory accesses for instructions which require it. For this purpose, we expose a few notable APIs to the user, allowing them to:

- Read from an address
- Write to an address
- Get a page
- Add a page

Read and write were the first obvious choice when it came to designing the interface, having a way to read up to word size (32bit, in this case) or write an arbitrary word value in the memory had a great deal of utility, it could be used for reading and writing registers and values to memory, but also allowed us to modify stack contents and restore stack frames after a call all by using the same 2 functions. Additionally, these methods handle permissions, meaning it will forbid writes to pages which are not mapped or writable, and throw an exception, for example.

Getting and adding pages was also extremely important, these are needed whenever code requests access to any address – in the read or write functions, for example. These functions navigate the page tables and return the page entry containing the physical address corresponding to the virtual address – if it exists. In the case of adding, the virtual address is then associated with a page and set of permissions specified. This is essential when setting up the memory space of the program prior to execution.

An alternative approach to reading and writing would be to allow any granularity requested and simply enable users to write to or read an arbitrary amount of data to a virtual address. This approach was evaluated, but need for this turned out to be scarce; we don't ever need to write large chunks of data into the address space during execution, as this is handled instruction by instruction and never more than a word at a time, as such it made sense to structure the memory API to mirror this.

Problems faced

With the scale and complexity of the project, several notable issues were unearthed during development that required workarounds worth mentioning.

The first, and most notable of these problems is the issue of system calls. The TRAP instruction on the mn103 allows assembly code to send an interrupt, which, at the OS level will be captured and may result in a system call being executed at that point. In the emulator environment we endeavoured to emulate CPU resources and memory, but did not touch system calls. This essentially means that any time a system call is executed in the emulator it is equivalent to a NOP. This was done as given scope and time available it was not deemed possible to provide system call emulation support as this would necessitate possibly implementing some/all system calls, in the interest of managing scope this was avoided. Logging of registers when executing a TRAP instruction is supported, however, and through this it is possible to understand what if anything the system call would be doing

Another problem encountered was finding a way to deal with file formats. The compiler produces an ELF binary, however owing to the lack of specialization in the emulator for any specific file format this made working with these files difficult. To solve the problem, we allow users to specify the execution start address, along with other parameters to customize the execution environment. This delegates the responsibility to the user but maximises the amount of control they have over the emulator.

Testing

Testing of the complete product uses fuzzing: Test cases are generated in the C language and are compiled for the host CPU (in this case, the CPU running on the desktop used to develop and test the emulator) and are cross compiled for the MN103. The test cases are then executed side by side, and the output “state”, is compared – the state consists of variables/registers after execution of the program.

To test the complete emulator, we also rely on a bank of premade c test cases from the c-testsuite repository [5] and some random testcases as mentioned. This is a collection of small single execution c programs that all return an expected value if all operations were successful. Most of the time this value will be 0. This makes testing our emulator extremely simple; cross compile the testcases, and run them in the emulator, then check if the final state of the register containing the main function return value is 0. If it is, we know that operations in the program were successful and consistent.

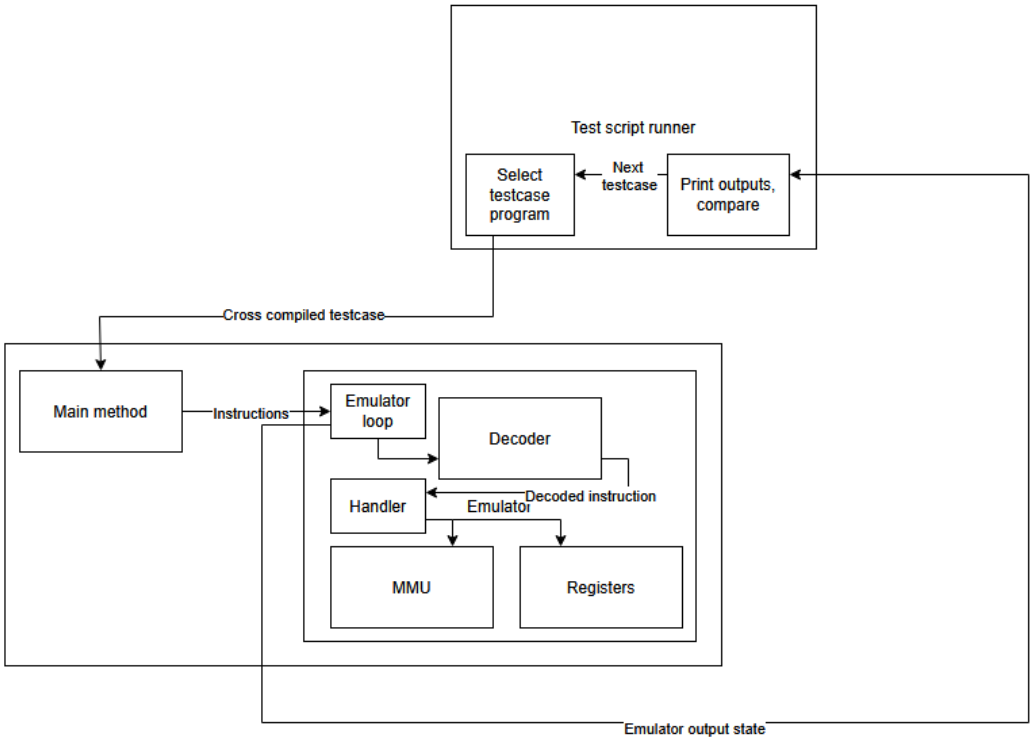


Figure 9: c-testsuite testing architecture

In addition to c-testsuite we also use the csmith [6] c code generator to generate some testcases for us. We minimize where necessary and modify these testcases, so the return value is gradually constructed by the operations in the program to obtain a useful output state. This is done through a simple XOR operation for select variables in the program.

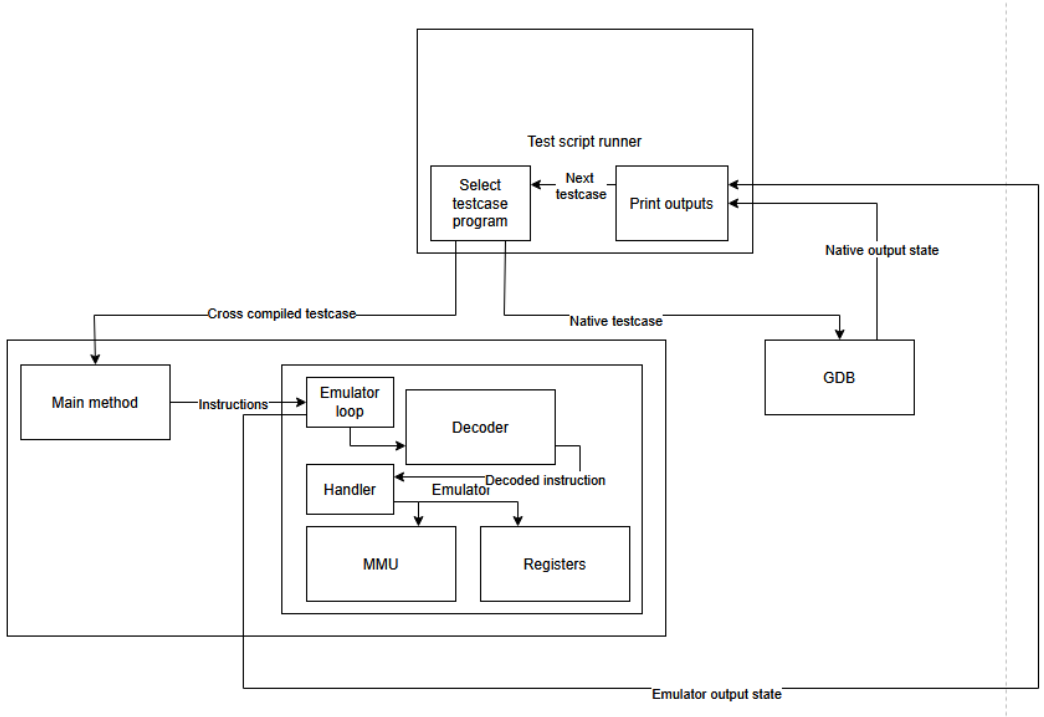


Figure 10: Csmith testing architecture

We are looking for places where the state would be different – IE, cases where the emulator differs from what is expected. We assume that regardless of which CPU the test case is executed on the output state should be identical, to enable this the code generated will be portable to ensure this is the case. Metrics such as speed is measured; the speed of the real CPU vs emulator is not a useful metric as in most cases the emulator will be slower, regardless of what program is running – however the speed of the emulator itself and number of instructions it is able to process per second is.

Based on the methods discussed the following test plan was assembled along with subsequent results:

Table 3: Test plan and results

ID	Name	Description	Expected result	Actual result
T01	Test suite samples	Running the emulator against a portion of samples from c-testsuite to ensure it can produce the correct output.	All tests produce the correct, expected value of 0.	Most tests (99/112) executed successfully.
T02	C smith samples	Running the emulator against a set of randomly generated programs.	All tests produce identical values to those present on the real CPU without issue and do not cause crashes or other errors.	All tests run successfully without error, with some producing identical results. Some have different results which necessitate fixes.
T03	Decoder unit tests	Running decoder portions related to each chunk of instructions and verifying the output against the specification.	Decoder unit tests will uncover problems and mistakes related to each given chunk of instructions.	Inconsistencies were uncovered and necessitated fixes.
T04	Emulator unit tests	Running emulator portions related to each chunk of instructions and verifying the output against the specification.	Emulator unit tests will uncover problems and inconsistencies.	Inconsistencies were uncovered and fixes were required.
T05	MMU unit tests	Tests performed after completion of initial MMU component to ensure it behaved as expected.	All tests execute and don’t cause errors.	Some tests were successful, however others caused errors which necessitated fixes.
T06	Fuzz tests	Program will be compiled with AFL++ instrumentation	No crashes will be encountered, and the program will handle	1 crash was encountered after 14 hours of fuzzing,

		and fuzzed for periods of time.	arbitrary user input correctly.	performance was bottlenecked by the number of hangs the fuzzer encountered.

Examples of outputs states for T01, T02, T05 and T06 can be found in appendices A, B, C, and D respectively.

All in all, 99/112 testcases from the c-testsuite repository were successful, and 10/10 of the random cases were successful, giving us a total accuracy of ~89% (((99+10) / (112+10)) * 100). This was quite a bit better than expected. The main thing causing cases to fail was GCC placing the start of data and strings immediately after the end of instructions in the text section in the binary's layout. This was problematic as we incorrectly assumed the text and other sections would have a significant margin of separation, and when loading the program, we loaded everything at the same address without this separation, meaning any memory accesses immediately after our text section would just access whatever came next in the file rather than the BSS. This was often a GCC version string, causing registers to contain unexpected bytes.

To remedy this problem the ability to specify where the BSS section was meant to start in the file was added, and if specified we inject a sequence of null bytes to act as the BSS section. This is admittedly a frail solution, however given that the project is only concerned with implementing the emulator and CPU logic and not parsing every executable format the emulator is presented with it wasn't deemed a big enough priority.

Since we intended to fuzz pure instruction behaviour, no C library was compiled. This meant that headers and functions inserted by the fuzzer (csmith) had to be replaced as we have no facility to provide these. This turned out to be incredibly simple as the generated samples relied on only a single header file which was easily replaced with a simple stub providing definitions for some integer types and some functions, one of which being the `_start` ELF entry point. This was necessary as there is no `_start` symbol provided when compiling `-nostdlib` with gcc, so for the mn103 samples we had to provide our own.

For the x86_64 samples we simply use the entry point provided, which, like our implementation, simply passes control to the `main` function.

Our speed in terms of IPS (instructions per second) can reach around 57 IPS, although this varies due to other factors and can go from around 35 thousand. If the output and logging was disabled and further optimization on top of what was already implemented was performed, we could potentially reach much higher, but as logging and debugging is a must, the output remains on. Below is a table with some experiments on the effect of deploying optimization techniques had on speed.

Table 4: Speed experiments

Description	Num of executions/s
GCC -O2 compiler flag	35066.508
GCC -O3 compiler flag	44872.340
GCC -O2 compiler flag + likely/unlikely	42626.564
GCC -O2 compiler flag + likely/unlikely + constexpr deployed in functions	57667.968
GCC -O3 compiler flag + likely/unlikely + constexpr	47983.748

All runs were testing on release versions. Interestingly it seems O3 was slower than O2 in some cases, this is a documented phenomenon [13].

In addition to the tests above, we deployed coverage guided fuzzing using AFL++ against the project. This is extremely simple to do, as all that is necessary is to change the compiler used to include instrumentation, then run the program inside afl-fuzz. On the first run, a corpus program is required so there is a starting point for mutations to build upon in the fuzzer. In this case I chose a simple program which computes the Fibonacci sequence as a starting point. A problem with this approach was the number of hangs encountered by the fuzzer; these significantly slowed progress and executions per second – but given the nature of the project this was impossible to avoid, as infinite loops and hanging is expected when running arbitrary instructions. Screenshots of fuzzing results can be found in appendix D.

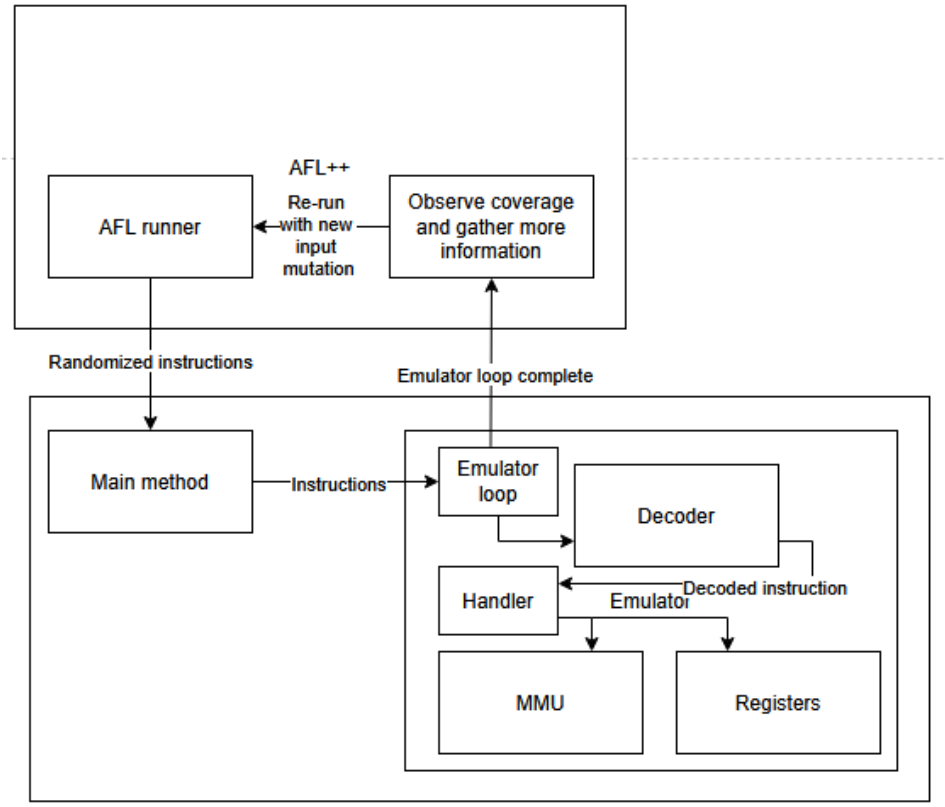


Figure 11: AFL++ Fuzzer structure

Chapter 4 - Evaluation

Evaluation of findings

Based on performance and accuracy measurements obtained from results, the product can consistently reproduce expected behaviour most of the time, and after a solution was provided in the form of a fix, it is now able to do so 100% of the time. This preserves the intent and behaviour of the original device in the manner which was desired. That said, complete device emulation is still a way away, as the emulator only reproduces instruction execution and memory. A better product would not only do this but also emulate hardware behaviour and other facets of the device. Despite this the product enables analysis of code and debugging of instructions, bypassing the need for the real CPU if this is the use case.

I believe that given a development process like my own, perhaps avoiding some of my pitfalls and accounting for time, perhaps also building in more detail as it relates to hardware a similar emulation

product could be created for most CPUs on the market, preserving their behaviour, as such this can be generalized quite far, all that's needed is a compiler, some online documentation and knowledge of emulation techniques.

Evaluation of the product

The project features handlers for every single instruction, having complete coverage which is supported across both emulator and decoder components. Testing of these components has revealed inconsistencies which have been fixed prior, although there may still be bugs and unexpected behaviours in decoder and emulator components for certain instructions. To help this, testing and testability of these components could have been better, a more reliable, scalable way to develop and deploy testcases would be preferred over the current set of limited testcases. That said, the emulator was still able to perform impressively and was consistent when presented with the gathered testcases after the fix mentioned prior was deployed.

Debugging and debuggability is good but could be better. Registers and memory are logged often, with the stack being dumped whenever instructions are executed that modify it, along with registers when they are loaded with different values. A way of specifying a region of memory to be logged in the emulator would be a desirable improvement, additionally some way to modify registers or memory would be a good addition, however this would probably require developing some kind scripting language for the emulator to make these modifications more specific – this would be even better as it would enable maximum control. An example of such a language is gdbscript which allows setting of breakpoints, logging, and other use of GDB's commands [14].

Performance is decent but could be improved in certain areas via implementation of caches for instruction decoding and something akin to a TLB for memory accesses. Doing this we would be able to store recently decoded instructions and recent address translations to prevent repeating these operations where they are unnecessary. In addition to this it's possible we could reduce the size of Instruction instances via removal of certain fields which could reduce memory usage. Multi-threading could also be used to speed up execution, with a decoder and emulator running in parallel with the emulator consuming decoded instructions and the decoder producing them in a queue like interface. Speed was not great in comparison with other emulators: ARMY, another open-source emulator for the ARM architecture can achieve speeds of hundreds of thousands of instructions per second [11], enabling logging slows this down significantly, however. This goes to show how much can be improved and optimized – and adding the ability for the user to select the level of logging they want would be another good improvement.

The emulator was designed with diverse and arbitrary user input in mind, with secure coding principles being followed; fields and segments of instructions are validated, and unsafe functions were avoided. A further improvement to be made could be running the coverage guided fuzzing against the emulator for more time, although 1 bug was found there may be more undiscovered.

Given the nature and inherent legality of clean room design the emulator breaks no laws and infringes on no copyright as the design was solely informed by existing open-source tools and manuals published in the public domain by Panasonic without knowledge of proprietary techniques.

The requirements developed have mostly been fulfilled, with small caveats in areas of capacity and reliability given its previous tendency to produce some incorrect outputs.

Project process

The development process was quite smooth. The schedule from the TOR document remained largely followed with a few hiccups later in the process as some bugs were encountered which stalled development progress. I've learned a great deal in this project, I believe it expanded my mental

horizons on understanding the vast number of possibilities which can arise during the process, good and bad. I have a much greater appreciation now of the planning and software development lifecycle than I did at the beginning.

More time should probably have been allocated for testing of the product, as this took a lot more than was expected to perform. There was a lot of testing variables which I failed to consider such as the way GCC generated binaries and having to build a way to capture program outputs on the emulator and native side.

On a technical level I've gained experience managing and maintaining a decently sized codebase, this has involved using build systems, maintaining structure and clean design and testing effectively – this resulted in a product which is robust and effective at what it seeks to do.

Chapter 5 - Conclusions and recommendations

One of the decisions made early in the project was to avoid touching emulating anything beyond the CPU and memory, this meant no system calls, no graphics, and no hardware peripherals, to name a few things. This was done to manage scope, as it would allow dedicating all available time to creating, testing and evaluating the CPU itself without being stretched too thin. Also, with the time allocated, the ability to provide emulation solutions for these aspects was doubted, however given more time on this project and perhaps in future it would be useful to provide support for some of these, especially graphics. In an ideal world eventually, the emulator would be able to run complex programs such as the Linux kernel. One of the more immediate improvements to be made is C library support. This would involve ensuring we are able to use many if not all the functions inside the standard library. This would be an essential building block for the above but would first require system call support.

In addition to this I would like to replicate the physical memory environment of the MN103 with the ability to further segment and manage it, for example being able to have a kernel which manages its own virtual address spaces within the emulator's virtual memory space. To tie in with this I would like to add more caching to the MMU to increase speed and efficiency of this component. This caching could be added to other areas such as the decoder to eliminate the need to decode the same instruction more than once. Despite the areas of development still required the product achieves the goal it set out to do; it preserves the behaviour of the CPU and documents its behaviour, it also enables analysis and execution of machine code.

Further work in the emulation domain could involve more documentation of the development process and techniques, especially at an academic level. Resources in this area are scarce, and it would benefit aspiring emulation developers and those with a similar ethos to this project if more such documents were publicised. As part of this, work could go toward assembling techniques relating to development of performant emulators, speed is a massive concern so having papers like [20] specifically in the domain of emulation discussing implementation of interpreters or dynamic translators would be extremely helpful for developers. Additionally, work could go towards documentation of the reverse engineering process/techniques for CPUs, a discussion on the optimal process for discerning instruction behaviour – perhaps through decomposing individual instructions into micro-operations would be useful for those trying to understand the innerworkings of a given CPU. Finally, more documenting of the testing process for emulators would be extremely useful, work focusing especially on testing the emulator against a core which isn't necessarily the target, but instead simply a trusted CPU, and still being able to determine correct /incorrect behaviour from that process. Such documentation would open the door to much more development as it eliminates the highest barrier to entry when trying to emulate a target, obtaining access to it.

References

[1] Moya Del Barrio, V. and Fernandez, A. (2001). *Study of the techniques for emulation programming (By a bored and boring guy)*. [online] Available at: http://www.xsim.com/papers/Bario.2001.emubook.pdf.

[2] Bellard, F. (2005). QEMU, a Fast and Portable Dynamic Translator. [online] Available at: https://www.usenix.org/legacy/event/usenix05/tech/freenix/full_papers/bellard/bellard.pdf.

[3] Martignoni, L., Paleari, R., Giampaolo Fresi Roglia and Bruschi, D. (2009). Testing CPU emulators. ISSTA '09. doi:https://doi.org/10.1145/1572272.1572303.

[4] Arpaci-Dusseau, R.H. and Arpaci-Dusseau, A.C. (2018). OPERATING SYSTEMS : three easy pieces. [online] S.L.: Createspace. Available at: https://pages.cs.wisc.edu/~remzi/OSTEP/.

[5] c-testsuite (2018). GitHub - c-testsuite/c-testsuite: A public database of C compiler test cases, minimal test runners, and public test results. [online] GitHub. Available at: https://github.com/c-testsuite/c-testsuite [Accessed 4 May 2025].

[7] dusek (2025). GitHub - dusek/army: ARM emulator written in C++. [online] GitHub. Available at: https://github.com/dusek/army [Accessed 4 May 2025].

[6] csmith-project (2024). GitHub - csmith-project/csmith: Csmith, a random generator of C programs. [online] GitHub. Available at: https://github.com/csmith-project/csmith.

[16] Fioraldi, A., Maier, D., Eißfeldt, H. and Heuse, M. (n.d.). AFL ++ : Combining Incremental Steps of Fuzzing Research. [online] Available at: https://www.usenix.org/system/files/woot20-paper-fioraldi.pdf.

[26] MN1030/MN103S Series Instruction Manual. (n.d.). Available at: https://bitsavers.org/components/panasonic/panaXseries/13250-040E_MN1030_MN103S_Series_Instruction_Manual.pdf [Accessed 4 May 2025].

[27] GCC team (2019). GCC, the GNU Compiler Collection - GNU Project - Free Software Foundation (FSF). [online] Gnu.org. Available at: https://gcc.gnu.org/.

Bibliography

[8] MAME project (2025). MAME | About MAME. [online] Mamedev.org. Available at: https://www.mamedev.org/about.html [Accessed 5 May 2025].

[9] Gametechwiki.com. (2023). Legal status and history of emulation - Emulation General Wiki. [online] Available at: https://emulation.gametechwiki.com/index.php/Legal_status_and_history_of_emulation [Accessed 4 May 2025].

[10] Dolphin Emulator. (n.d.). Dolphin Emulator - Frequently Asked Questions. [online] Available at: https://dolphin-emu.org/docs/faq/.

[11] dusek (2025). GitHub - dusek/army: ARM emulator written in C++. [online] GitHub. Available at: https://github.com/dusek/army [Accessed 4 May 2025].

[12] Retroreversing.com. (2022). RetroReversing. [online] Available at: https://www.retroreversing.com/clean-room-reversing [Accessed 5 May 2025].

[13] TechHara (2023). Compiler optimizations can be misleading - TechHara - Medium. [online] Medium. Available at: https://medium.com/@techhara/compiler-optimizations-can-be-tricky-ee323415e6a [Accessed 5 May 2025].

[14] sourceware.org. (n.d.). GDB: The GNU Project Debugger. [online] Available at: https://sourceware.org/gdb/.

[15] Intel (n.d.). Intel® 64 and IA-32 Architectures Software Developer Manuals. [online] Intel. Available at: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html.

[17] Citron project (2025). Citron - Nintendo Homebrew Emulator. [online] Citron-emu.org. Available at: https://citron-emu.org/ [Accessed 5 May 2025].

[18] Zhang, A. (2020). QEMU VM Escape Vulnerability (CVE-2020-14364) Threat Alert - NSFOCUS, Inc., a global network and cyber security leader, protects enterprises and carriers from advanced cyber attacks. [online] NSFOCUS, Inc., a global network and cyber security leader, protects enterprises and carriers from advanced cyber attacks. Available at: https://nsfocusglobal.com/qemu-vm-escape-vulnerability-cve-2020-14364-threat-alert/ [Accessed 5 May 2025].

[19] en.cppreference.com. (n.d.). constexpr specifier (since C++11) - cppreference.com. [online] Available at: https://en.cppreference.com/w/cpp/language/constexpr.

[20] Bilokon, P. and Gunduz, B. (2023). C++ Design Patterns for Low-latency Applications Including High-frequency Trading. [online] arXiv.org. Available at: https://arxiv.org/abs/2309.04259 [Accessed 4 May 2025].

[21] Uvic.ca. (2025). [dcl.attr]. [online] Available at: https://www.ece.uvic.ca/~frodo/cppbook/cppdraft/n4861/dcl.attr [Accessed 5 May 2025].

[22] Gitlab.io. (2025). Security — QEMU documentation. [online] Available at: https://qemu-project.gitlab.io/qemu/system/security.html [Accessed 4 May 2025].

[23] cvedetails (2023). CVE-2023-6693 : A stack based buffer overflow was found in the virtio-net device of QEMU. This i. [online] Cvedetails.com. Available at: https://www.cvedetails.com/cve/CVE-2023-6693/ [Accessed 5 May 2025].

[24] cvedetails (2024). CVE-2024-6519 : A use-after-free vulnerability was found in the QEMU LSI53C895A SCSI Host Bus Ad. [online] Cvedetails.com. Available at: https://www.cvedetails.com/cve/CVE-2024-6519/ [Accessed 5 May 2025].

[25] Qemu.org. (2025). Secure Coding Practices — QEMU documentation. [online] Available at: https://www.qemu.org/docs/master/devel/secure-coding-practices.html [Accessed 4 May 2025].

[26] Lkml.org. (2018). LKML: Arnd Bergmann: [GIT PULL] arch: remove obsolete architecture ports. [online] Available at: https://lkml.org/lkml/2018/4/2/72 [Accessed 4 May 2025].

Appendices

A

C testsuite run_tests script output

```
D0      :00000000

D0      :00000000
```

```
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
```

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000001

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :4728203a

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :203a4343

D0 :4728203a

D0 :4728203a

D0 :4728203a

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000001

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

D0 :00000000

```
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :00000000
D0      :4728203a
D0      :00000000
D0      :00000000
```

B

Run_tests script output – csmith generated testcases:

```
1_sim
1_ntv
Simulated: D0  :00000517
Real:  returned: 00000517
2_sim
2_ntv
Simulated: D0  :23db24ef
Real:  returned: 23db24ef
3_sim
3_ntv
Simulated: D0  :fffffffc
Real:  returned: fffffffc
4_sim
4_ntv
Simulated: D0  :b60fb25e
Real:  returned: b60fb25e
5_sim
```

```
5_ntv
Simulated: D0  :000000df
Real:  returned: 000000df
6_sim
6_ntv
Simulated: D0  :00000000
Real:  returned: 00000000
7_sim
7_ntv
Simulated: D0  :9f799d5a
Real:  returned: 9f799d5a
8_sim
8_ntv
Simulated: D0  :3388fe33
Real:  returned: 3388fe33
9_sim
9_ntv
Simulated: D0  :00007765
Real:  returned: 00007765
10_sim
10_ntv
Simulated: D0  :512bd689
Real:  returned: 512bd689
```

C

```
Sizeof(page_entry) -> 6
V page: 7ffff000
PLD idx: 1
PMD idx: 1ff
PTE idx: 1ff
GET------
PLD idx: 1
```

PMD idx: 1ff

PTE idx: 1ff

Physical addr: 0x625000006000

MMU::write writing: 4141414142 --> 7ffffff8

PLD idx: 1

PMD idx: 1ff

PTE idx: 1ff

MMU::write Page address: 625000006

MMU::read reading: 7ffffff8

PLD idx: 1

PMD idx: 1ff

PTE idx: 1ff

MMU::read Page address: 625000006

Value at 7ffffff8 -> 4141414142

Testing cross page reads and writes...

V page: 1000

PLD idx: 0

PMD idx: 0

PTE idx: 1

V page: 2000

PLD idx: 0

PMD idx: 0

PTE idx: 2

V page: 14000

PLD idx: 0

PMD idx: 0

PTE idx: 14

V page: 41414000

PLD idx: 1

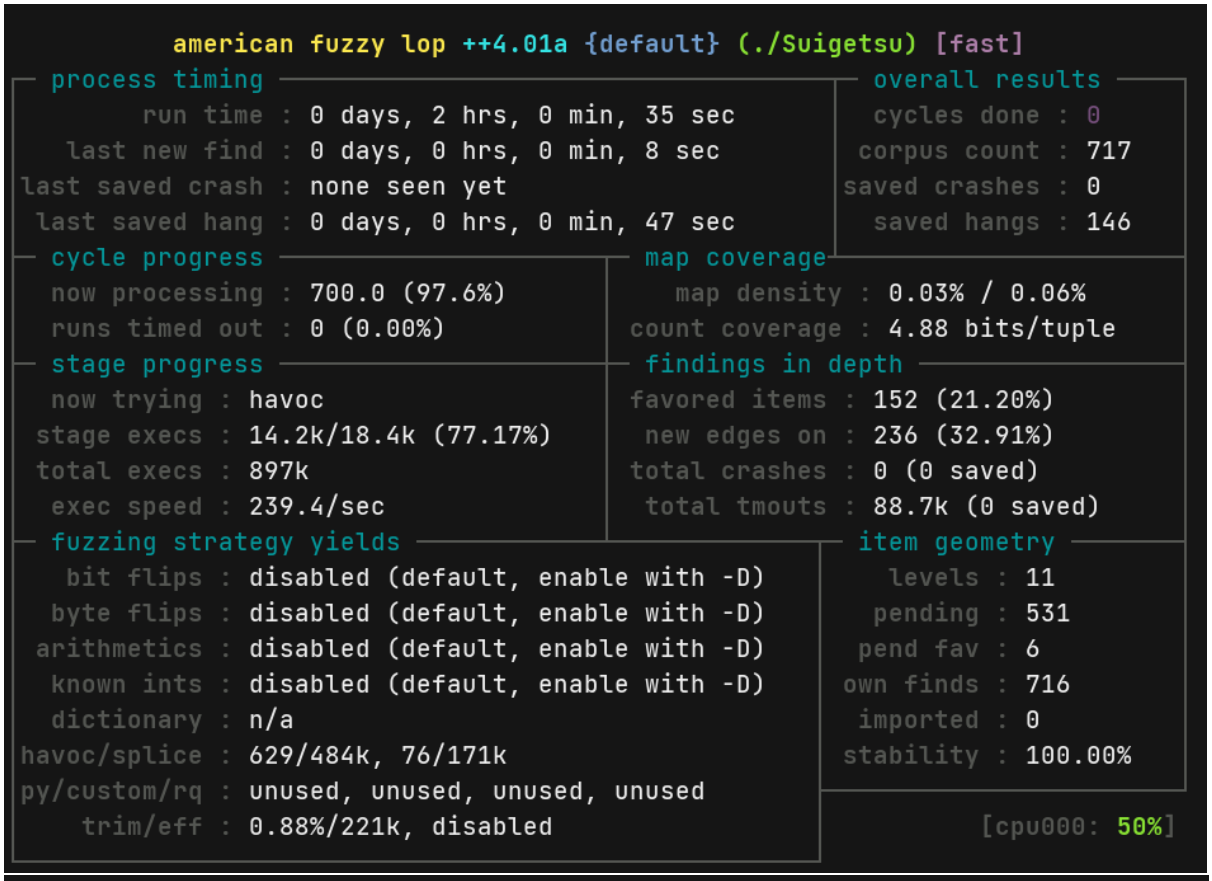
PMD idx: a

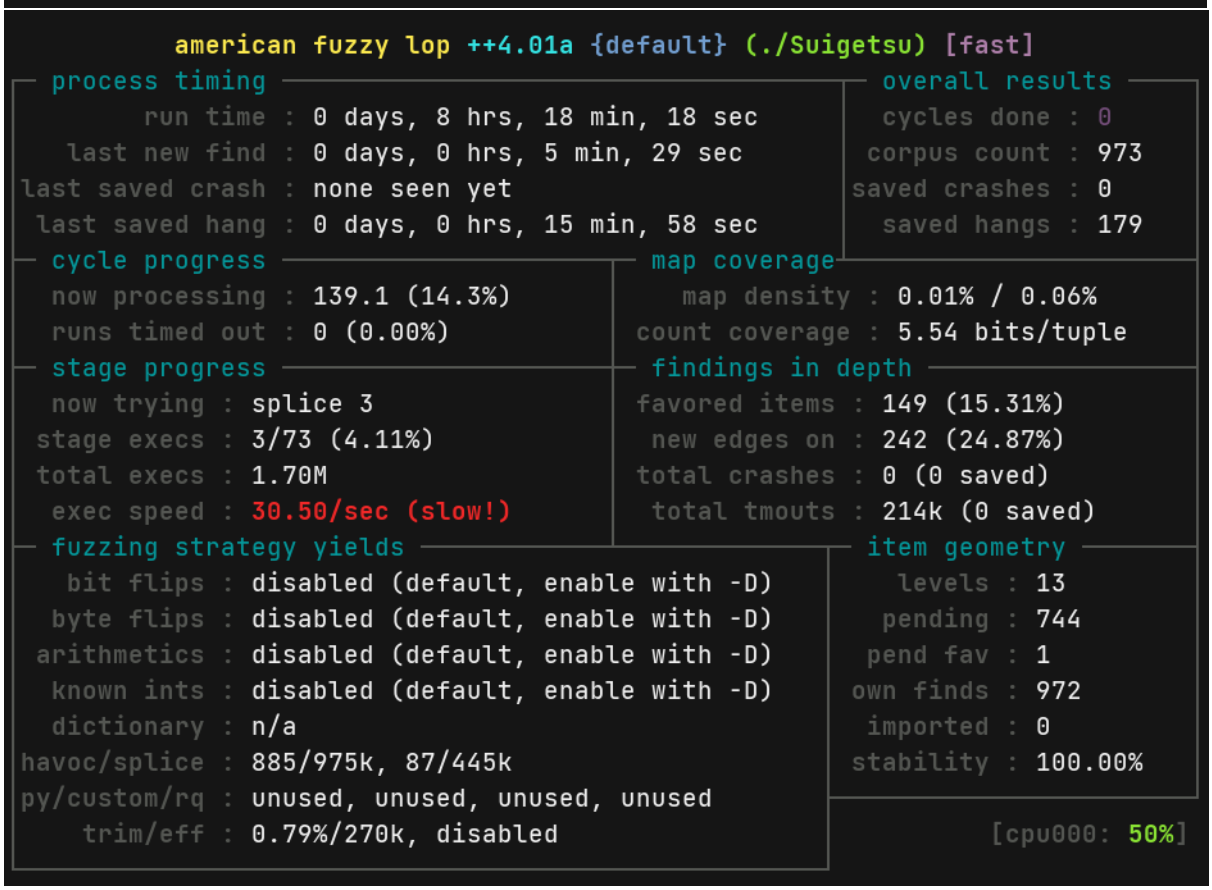
PTE idx: 14

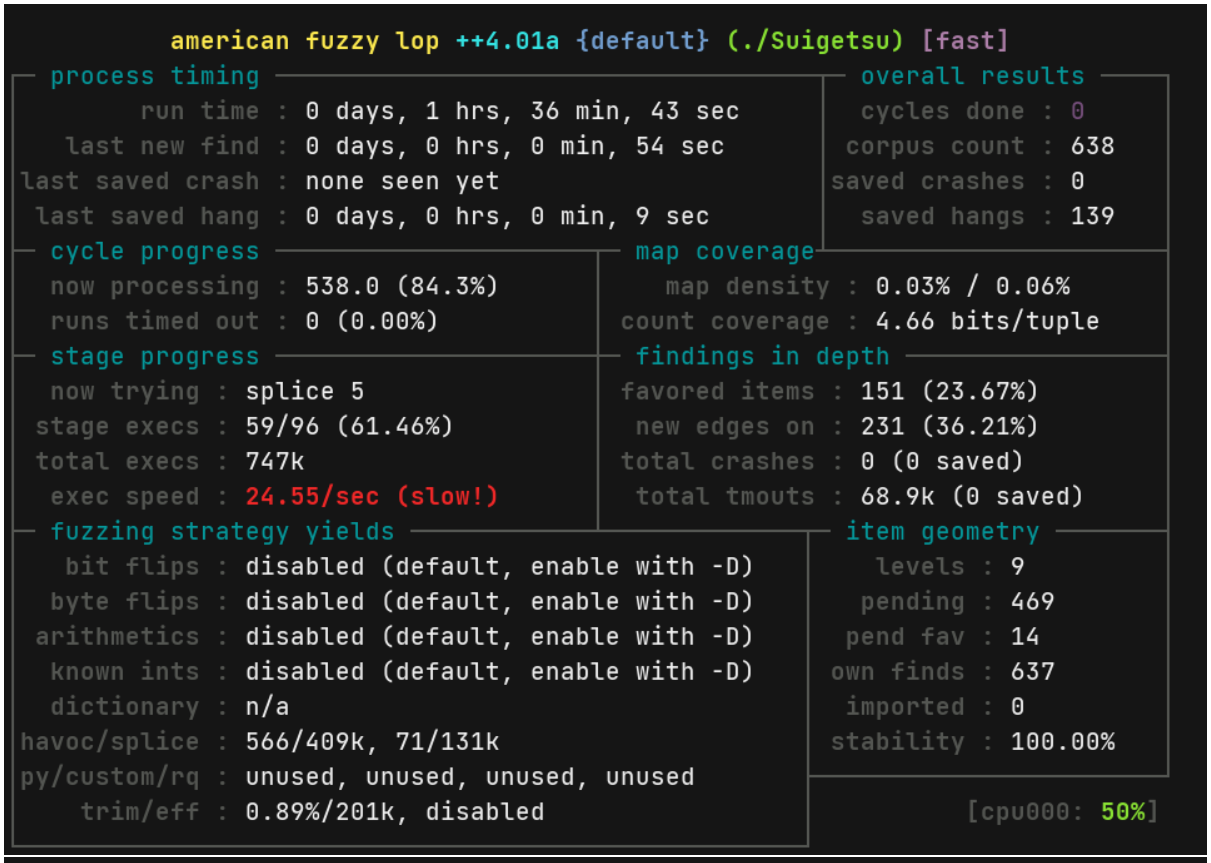
```
PLD idx: 0
PMD idx: 0
PTE idx: 14
p1 addr: 0x625000012000
PLD idx: 1
PMD idx: a
PTE idx: 14
p2 addr: 0x625000017000
```

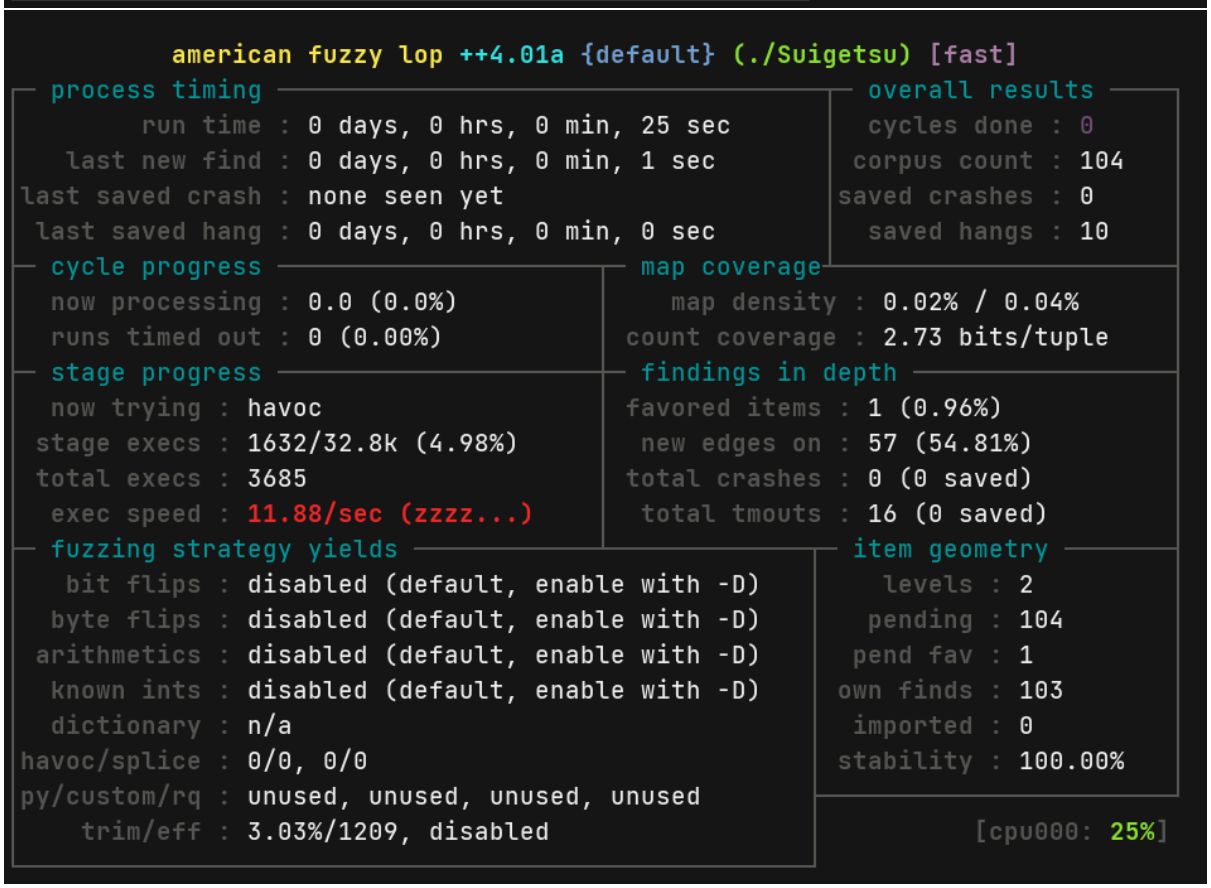
D

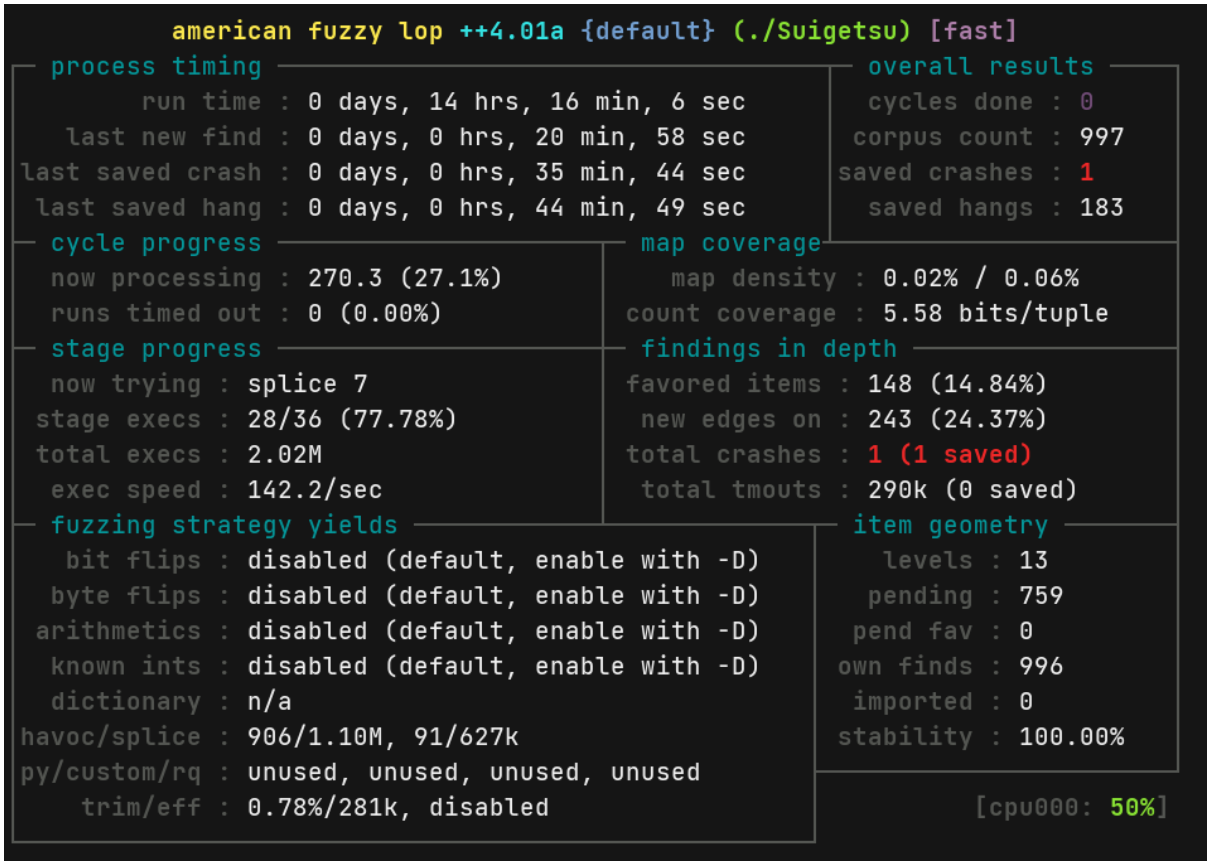
Fuzzing results screenshots


american fuzzy lop ++4.01a {default} (./Suigetsu) [fast]
process timing
run time : 0 days, 2 hrs, 0 min, 35 sec
last new find : 0 days, 0 hrs, 0 min, 8 sec
last saved crash : none seen yet
last saved hang : 0 days, 0 hrs, 0 min, 47 sec
overall results
cycles done : 0
corpus count : 717
saved crashes : 0
saved hangs : 146
cycle progress
now processing : 700.0 (97.6%)
runs timed out : 0 (0.00%)
map coverage
map density : 0.03% / 0.06%
count coverage : 4.88 bits/tuple
stage progress
now trying : havoc
stage execs : 14.2k/18.4k (77.17%)
total execs : 897k
exec speed : 239.4/sec
findings in depth
favored items : 152 (21.20%)
new edges on : 236 (32.91%)
total crashes : 0 (0 saved)
total tmouts : 88.7k (0 saved)
fuzzing strategy yields
bit flips : disabled (default, enable with -D)
byte flips : disabled (default, enable with -D)
arithmetics : disabled (default, enable with -D)
known ints : disabled (default, enable with -D)
dictionary : n/a
havoc/splice : 629/484k, 76/171k
py/custom/rq : unused, unused, unused, unused
trim/eff : 0.88%/221k, disabled
item geometry
levels : 11
pending : 531
pend fav : 6
own finds : 716
imported : 0
stability : 100.00%
[cpu000: 50%]


american fuzzy lop ++4.01a {default} (./Suigetsu) [fast]
process timing
run time : 0 days, 8 hrs, 18 min, 18 sec
last new find : 0 days, 0 hrs, 5 min, 29 sec
last saved crash : none seen yet
last saved hang : 0 days, 0 hrs, 15 min, 58 sec
overall results
cycles done : 0
corpus count : 973
saved crashes : 0
saved hangs : 179
cycle progress
now processing : 139.1 (14.3%)
runs timed out : 0 (0.00%)
map coverage
map density : 0.01% / 0.06%
count coverage : 5.54 bits/tuple
stage progress
now trying : splice 3
stage execs : 3/73 (4.11%)
total execs : 1.70M
exec speed : 30.50/sec (slow!)
findings in depth
favored items : 149 (15.31%)
new edges on : 242 (24.87%)
total crashes : 0 (0 saved)
total tmouts : 214k (0 saved)
fuzzing strategy yields
bit flips : disabled (default, enable with -D)
byte flips : disabled (default, enable with -D)
arithmetics : disabled (default, enable with -D)
known ints : disabled (default, enable with -D)
dictionary : n/a
havoc/splice : 885/975k, 87/445k
py/custom/rq : unused, unused, unused, unused
trim/eff : 0.79%/270k, disabled
item geometry
levels : 13
pending : 744
pend fav : 1
own finds : 972
imported : 0
stability : 100.00%
[cpu000: 50%]


american fuzzy lop ++4.01a {default} (./Suigetsu) [fast]
process timing
run time : 0 days, 1 hrs, 36 min, 43 sec
last new find : 0 days, 0 hrs, 0 min, 54 sec
last saved crash : none seen yet
last saved hang : 0 days, 0 hrs, 0 min, 9 sec
overall results
cycles done : 0
corpus count : 638
saved crashes : 0
saved hangs : 139
cycle progress
now processing : 538.0 (84.3%)
runs timed out : 0 (0.00%)
map coverage
map density : 0.03% / 0.06%
count coverage : 4.66 bits/tuple
stage progress
now trying : splice 5
stage execs : 59/96 (61.46%)
total execs : 747k
exec speed : 24.55/sec (slow!)
findings in depth
favored items : 151 (23.67%)
new edges on : 231 (36.21%)
total crashes : 0 (0 saved)
total tmouts : 68.9k (0 saved)
fuzzing strategy yields
bit flips : disabled (default, enable with -D)
byte flips : disabled (default, enable with -D)
arithmetics : disabled (default, enable with -D)
known ints : disabled (default, enable with -D)
dictionary : n/a
havoc/splice : 566/409k, 71/131k
py/custom/rq : unused, unused, unused, unused
trim/eff : 0.89%/201k, disabled
item geometry
levels : 9
pending : 469
pend fav : 14
own finds : 637
imported : 0
stability : 100.00%
[cpu000: 50%]


american fuzzy lop ++4.01a {default} (./Suigetsu) [fast]
process timing
run time : 0 days, 0 hrs, 0 min, 25 sec
last new find : 0 days, 0 hrs, 0 min, 1 sec
last saved crash : none seen yet
last saved hang : 0 days, 0 hrs, 0 min, 0 sec
overall results
cycles done : 0
corpus count : 104
saved crashes : 0
saved hangs : 10
cycle progress
now processing : 0.0 (0.0%)
runs timed out : 0 (0.00%)
map coverage
map density : 0.02% / 0.04%
count coverage : 2.73 bits/tuple
stage progress
now trying : havoc
stage execs : 1632/32.8k (4.98%)
total execs : 3685
exec speed : 11.88/sec (zzzz...)
findings in depth
favored items : 1 (0.96%)
new edges on : 57 (54.81%)
total crashes : 0 (0 saved)
total tmouts : 16 (0 saved)
fuzzing strategy yields
bit flips : disabled (default, enable with -D)
byte flips : disabled (default, enable with -D)
arithmetics : disabled (default, enable with -D)
known ints : disabled (default, enable with -D)
dictionary : n/a
havoc/splice : 0/0, 0/0
py/custom/rq : unused, unused, unused, unused
trim/eff : 3.03%/1209, disabled
item geometry
levels : 2
pending : 104
pend fav : 1
own finds : 103
imported : 0
stability : 100.00%
[cpu000: 25%]


american fuzzy lop ++4.01a {default} (./Suigetsu) [fast]
process timing
run time : 0 days, 14 hrs, 16 min, 6 sec
last new find : 0 days, 0 hrs, 20 min, 58 sec
last saved crash : 0 days, 0 hrs, 35 min, 44 sec
last saved hang : 0 days, 0 hrs, 44 min, 49 sec
overall results
cycles done : 0
corpus count : 997
saved crashes : 1
saved hangs : 183
cycle progress
now processing : 270.3 (27.1%)
runs timed out : 0 (0.00%)
map coverage
map density : 0.02% / 0.06%
count coverage : 5.58 bits/tuple
stage progress
now trying : splice 7
stage execs : 28/36 (77.78%)
total execs : 2.02M
exec speed : 142.2/sec
findings in depth
favored items : 148 (14.84%)
new edges on : 243 (24.37%)
total crashes : 1 (1 saved)
total tmouts : 290k (0 saved)
fuzzing strategy yields
bit flips : disabled (default, enable with -D)
byte flips : disabled (default, enable with -D)
arithmetics : disabled (default, enable with -D)
known ints : disabled (default, enable with -D)
dictionary : n/a
havoc/splice : 906/1.10M, 91/627k
py/custom/rq : unused, unused, unused, unused
trim/eff : 0.78%/281k, disabled
item geometry
levels : 13
pending : 759
pend fav : 0
own finds : 996
imported : 0
stability : 100.00%
[cpu000: 50%]